Universidad Autónoma de la Ciudad de México

Licenciatura en Ingeniería de Software

Arquitectura de Computadoras

Práctica 02

Promedio de Números Enteros con Signo en Ensamblador x86

Alumno: Edson Joel Carrera Avila

Grupo: 302

Profesor: Prof. Mtro. Omar Nieto Crisóstomo

Fecha: 29 de abril de 2026

Índice

1. Introducción **2**

2. Organización del programa **2**

3. Código fuente **3**

3.1. Código completo (promedio.asm) 3

4. Instrucciones x86 utilizadas **4**

5. El rol de los registros **5**

6. Ejemplo de ejecución **6**

7. Repositorio GitHub **7**

8. Conclusiones **7**

1. Introducción

Esta práctica consiste en escribir un programa en **lenguaje ensamblador x86** que tome un arreglo de números enteros con signo almacenado en memoria y calcule su promedio aritmético, realizando todas las operaciones directamente sobre los registros del procesador.

El objetivo principal no es simplemente "calcular un promedio", sino comprender cómo una computadora realiza aritmética sobre enteros con signo a su nivel más básico. Para lograrlo, es necesario entender tres conceptos fundamentales: la **representación de enteros con signo**, la **división entera con signo** y el **recorrido de arreglos en memoria**.

¿Qué es un entero con signo (SDWORD)?

Las computadoras representan números negativos utilizando el sistema de **complemento a dos**. En x86, un `SDWORD` (*Signed Double Word*) es un entero de 32 bits con signo, capaz de representar valores en el rango $[-2^{31},\ 2^{31}-1]$, es decir, desde $-2\,147\,483\,648$ hasta $2\,147\,483\,647$.

¿Por qué se necesita CDQ antes de dividir?

La instrucción `IDIV` (división entera con signo) espera un dividendo de **64 bits** repartido entre dos registros: `EDX` (32 bits altos) y `EAX` (32 bits bajos). Si la suma está en `EAX` y se divide directamente, `EDX` puede contener basura y producir resultados incorrectos o provocar una excepción de overflow. La instrucción `CDQ` (*Convert Double to Quad*) extiende el bit de signo de `EAX` hacia todo `EDX`, construyendo correctamente ese número de 64 bits:

$$\text{EDX:EAX} \xrightarrow{\texttt{IDIV}\ n} \quad \text{EAX} = \left\lfloor \frac{\text{suma}}{n} \right\rfloor, \quad \text{EDX} = \text{suma} \bmod n \tag{1}$$

2. Organización del programa

El proyecto consta de un único archivo:

Archivo	Qué hace
`promedio.asm`	Declara el arreglo de enteros con signo en memoria, los recorre acumulando su suma y calcula el promedio y el residuo mediante división entera con signo.

Cuadro 1: Archivo del proyecto

El programa se divide internamente en dos secciones:

- **Sección `.data`:** declara el arreglo `Dato` con seis enteros con signo, calcula automáticamente la cantidad de elementos mediante la directiva `EQU`, y reserva espacio para las variables `Promedio` y `Residuo`.
- **Sección `.code`:** contiene las instrucciones que el procesador ejecuta para recorrer el arreglo, sumar todos sus elementos y dividir entre la cantidad de elementos.

El flujo general del programa es el siguiente:

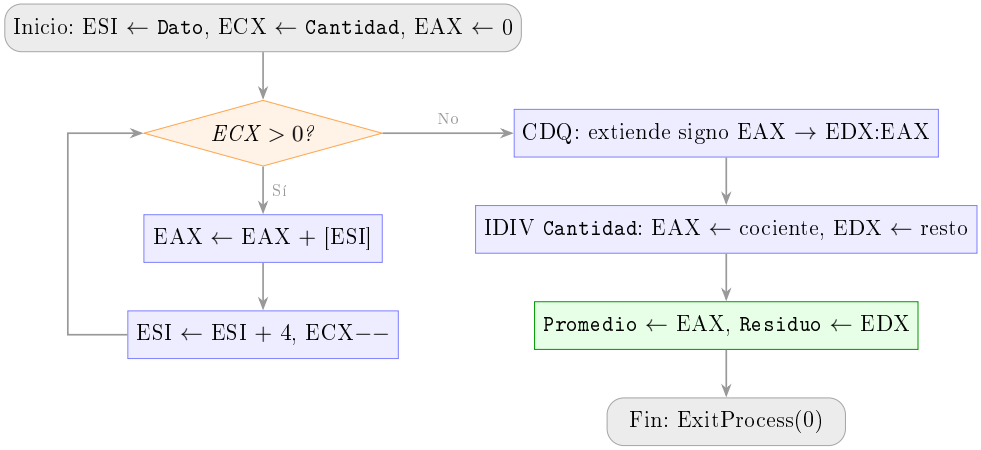

Figura 1: Diagrama de flujo del algoritmo de cálculo del promedio

1. El programa inicializa `ESI` apuntando al primer elemento del arreglo, `ECX` con la cantidad de elementos y `EAX` en cero (acumulador de la suma).
2. En cada iteración, suma el valor apuntado por `ESI` a `EAX` y avanza el puntero 4 bytes (tamaño de un `SDWORD`).
3. La instrucción `LOOP` decrementa `ECX` automáticamente y repite mientras sea mayor que cero.
4. Al salir del ciclo, `CDQ` extiende el signo de `EAX` hacia `EDX` para preparar la división de 64 bits.
5. `IDIV` divide `EDX:EAX` entre la cantidad de elementos; el cociente queda en `EAX` y el resto en `EDX`.
6. Ambos resultados se guardan en las variables `Promedio` y `Residuo` definidas en `.data`.

3. Código fuente

3.1. Código completo (promedio.asm)

A continuación se presenta el programa completo:

```
.586
.model flat, stdcall

.data
    Dato       SDWORD  -10, 20, -30, 40, -50, 60
    ; Calcula la cantidad de elementos: (Direccion actual - Inicio de Dato) / 4 bytes
    Cantidad   EQU ($ - Dato) / 4
    Promedio   SDWORD ?
    Residuo    SDWORD ?

.code
EXTERN ExitProcess@4 : PROC

start:
    lea esi, Dato        ; Carga la direccion de memoria inicial del arreglo en ESI
    mov ecx, Cantidad    ; Inicializa el contador del bucle con el numero de elementos
    xor eax, eax         ; Limpia EAX (lo pone en 0) para usarlo como acumulador

sumar:
    add eax, [esi]       ; Suma el valor apuntado por ESI al acumulador EAX
    add esi, 4           ; Mueve el puntero ESI al siguiente elemento (4 bytes adelante)
    loop sumar           ; Decrementa ECX y salta a sumar si ECX > 0

    cdq                  ; Extiende el signo de EAX hacia EDX (crea EDX:EAX de 64 bits)
    mov ebx, Cantidad    ; Carga el divisor (n=6) en EBX
    idiv ebx             ; Divide EDX:EAX entre EBX. Cociente en EAX, Resto en EDX

    mov Promedio, eax    ; Guarda el promedio (cociente)
    mov Residuo, edx     ; Guarda el residuo de la division

    push 0
    call ExitProcess@4
end start
```

Listing 1: promedio.asm — Promedio de enteros con signo

4. Instrucciones x86 utilizadas

La siguiente tabla resume todas las instrucciones ensamblador empleadas en el programa, con una explicación accesible de su función:

Instrucción	Qué hace en términos simples
`LEA`	Carga en un registro la *dirección* de una variable en memoria (no su valor).
`MOV`	Copia un valor de un lugar a otro (entre registros o entre registro y memoria).
`XOR`	Aplica la operación lógica OR-exclusivo; usada aquí para poner `EAX` en cero de forma eficiente.
`ADD`	Suma el operando fuente al destino y guarda el resultado en el destino.
`LOOP`	Decrementa `ECX` en 1 y salta a la etiqueta indicada si `ECX` $\neq 0$; equivale a un `for` con contador.
`CDQ`	Extiende el bit de signo de `EAX` hacia todo `EDX`, formando un entero de 64 bits en `EDX:EAX`.
`IDIV`	Divide el entero de 64 bits en `EDX:EAX` entre el operando dado; el cociente queda en `EAX` y el resto en `EDX`.
`PUSH`	Coloca un valor en la pila de memoria.
`CALL`	Llama a una función o procedimiento externo.

Cuadro 2: Instrucciones x86 utilizadas en el programa

5. El rol de los registros

En este programa intervienen varios registros del procesador, cada uno con una responsabilidad específica:

- **ESI** (*Extended Source Index*): actúa como **puntero** al elemento actual del arreglo. Se inicializa con la dirección de `Dato` y avanza 4 bytes en cada iteración.
- **ECX** (*Extended Counter Register*): registro contador empleado por la instrucción `LOOP`. Se inicializa con la cantidad de elementos y se decrementa automáticamente en cada vuelta.
- **EAX** (*Extended Accumulator Register*): acumula la suma de todos los elementos durante el ciclo y, tras la división, contiene el **cociente** (promedio).
- **EDX** (*Extended Data Register*): recibe la extensión de signo de `EAX` mediante `CDQ` y, tras la división, contiene el **resto**.
- **EBX** (*Extended Base Register*): almacena el divisor (`Cantidad`) para la instrucción `IDIV`.

La variable `Cantidad` se calcula en **tiempo de ensamblado** mediante la directiva `EQU` y la expresión `($ - Dato) / 4`, donde `$` representa la posición actual en memoria. Esto evita actualizar manualmente el contador si se modifica el arreglo.

6. Ejemplo de ejecución

El arreglo declarado en `.data` es:

$$\{-10,\ 20,\ -30,\ 40,\ -50,\ 60\}$$

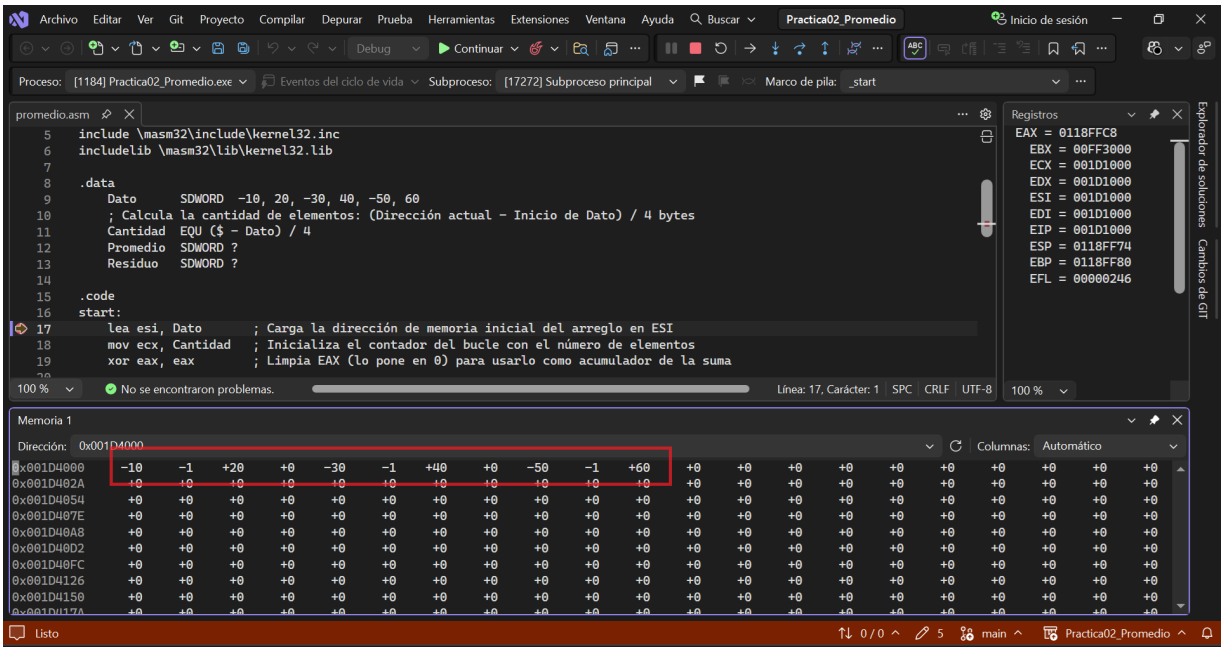

Figura 2: Estado inicial: ESI apunta a `Dato`, ECX = 6, EAX = 0

Iteración	Elemento [ESI]	EAX (acumulador)	ECX restante
1	-10	-10	5
2	$+20$	$+10$	4
3	-30	-20	3
4	$+40$	$+20$	2
5	-50	-30	1
6	$+60$	$+30$	0

Cuadro 3: Evolución del acumulador durante el ciclo `sumar`

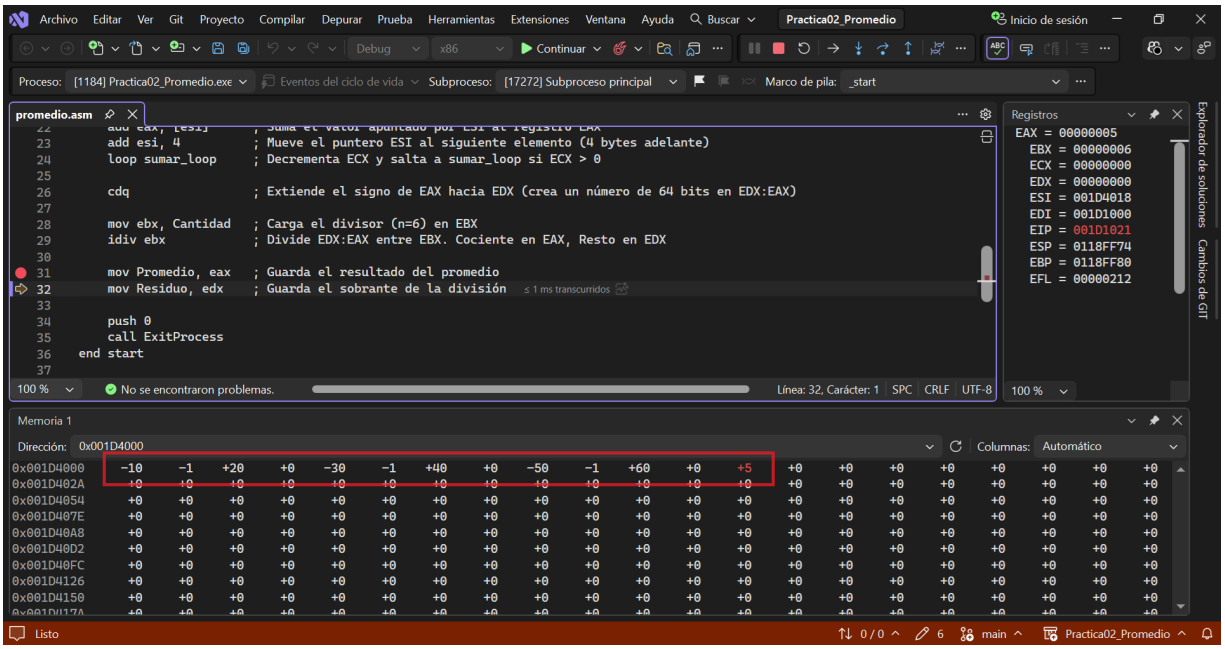

Figura 3: Ventana de memoria: `Promedio = 0x05`, `Residuo = 0x00` escritos en `.data`

Variable	Registro origen	Valor almacenado
`Promedio`	`EAX`	5
`Residuo`	`EDX`	0

Cuadro 4: Valores escritos en `.data` al finalizar el programa

7. Repositorio GitHub

`https://github.com/7mo-ArquitecturaComputadoras/Practica02_Promedio`

8. Conclusiones

- El programa demuestra que **el procesador no distingue intrínsecamente entre suma y promedio**: el promedio no es más que una suma seguida de una división, y ambas operaciones se realizan directamente sobre registros.
- La instrucción `CDQ` es indispensable antes de `IDIV`: sin ella, `EDX` contendría valores arbitrarios y el resultado de la división sería incorrecto o provocaría una excepción, especialmente con sumas negativas.
- La directiva `EQU` con la expresión `($ - Dato) / 4` es un patrón idiomático en MASM para calcular el tamaño de un arreglo **en tiempo de ensamblado**, eliminando la posibilidad de errores al modificar la lista de datos.

- El uso de `LOOP` con `ECX` como contador ilustra cómo el hardware ofrece soporte nativo para estructuras de control como los ciclos `for` de los lenguajes de alto nivel.
- Esta práctica evidencia la cadena completa de operaciones que ejecuta un procesador cuando un lenguaje como Python o Java invoca `statistics.mean()`: en el fondo, el hardware realiza exactamente esta misma secuencia de sumas, extensión de signo y división con signo.